

Prueba DEV 6 - Fanz CLI para agentes

Objetivo: construir un CLI usable por una persona y por agentes de IA para operar una cuenta mock de una ticketera. El entregable debe poder probarse desde una URL pública, sin descargar ni instalar nada.

La prueba no debe usar datos reales. Inventá una cuenta mock con eventos, fechas, tickets, descuentos, ventas y compradores suficientes para demostrar el producto.

Desafío

- Crear un CLI de ticketing básico que permita autenticar una cuenta mock y ejecutar acciones sobre esa cuenta.
- Diseñar comandos que un agente de IA pueda llamar de forma confiable, con outputs claros, estables y preferentemente parseables en JSON.
- Incluir un instructivo corto para que podamos copiar comandos y probar el flujo completo sin pedirte ayuda.

Funcionalidades mínimas esperadas

- Autenticación: una forma simple y documentada de iniciar sesión, seleccionar cuenta o usar un token mock.
- Eventos: crear un evento básico con nombre, descripción, ubicación, fecha(s) y estado de venta.
- Fechas: crear, editar, listar y borrar fechas o funciones de un evento.
- Tickets: crear, editar, listar y borrar tipos de ticket, con precio, stock, cupo vendido y estado.
- Descuentos: crear, editar, listar y borrar códigos o reglas simples de descuento.
- Ventas: consultar ventas mock, compradores, tickets emitidos, revenue, stock restante y estado de una orden.
- Acciones útiles: incluir al menos una acción extra que tenga sentido para una ticketera, por ejemplo enviar/re-enviar tickets a un email mock, pausar ventas, duplicar evento o exportar ventas.

Cómo deberíamos poder probarlo

- Entrar a una URL pública y usar el CLI desde el navegador, una web terminal, un playground o una interfaz equivalente.
- Seguir un instructivo con credenciales/token mock y ejemplos de comandos end-to-end.
- Crear un evento desde cero, modificarlo, agregar tickets/descuentos, consultar ventas y ejecutar una acción sobre la cuenta mock.
- Ver errores útiles cuando un comando es inválido, ambiguo o intenta hacer algo riesgoso.

Ejemplos orientativos, no obligatorios

```
fanz login --token mock_admin
fanz events create --name "Fiesta Demo" --date 2026-07-20 --ticket "General:10000:500" --json
fanz tickets update EVT_123 TCK_1 --price 12000 --json
fanz sales list --event EVT_123 --json
fanz discounts create --event EVT_123 --code DEMO20 --percent 20 --json
```

Guardrails que suman puntos

- Modo dry-run o preview antes de acciones destructivas.
- Confirmaciones, validaciones de negocio y mensajes de error accionables.
- Audit log simple de comandos ejecutados.
- Separar comandos de lectura y escritura, y dejar claro qué permisos tiene cada token mock.

Qué vamos a evaluar

- Autonomía para decidir el scope correcto sin pedir instrucciones excesivas.
- Calidad de la interfaz para agentes: comandos claros, outputs predecibles y bajo riesgo de acciones accidentales.
- Capacidad de modelar una ticketera simple con buenas abstracciones.
- Calidad técnica general, deploy, documentación, UX de prueba y manejo de casos raros.

Entregable: una URL pública, credenciales/token mock, instructivo breve de uso, comandos de ejemplo y una nota corta con decisiones, supuestos y limitaciones.